FORECASTING PRECIPITATION AND TEMPERATURE EVOLUTION PATTERNS UNDER CLIMATE CHANGE USING A RANDOM FOREST APPROACH WITH SEASONAL BIAS CORRECTION

[1]S SHREENIDHI, AIML, [2]SHREYA RANI JOSEPH, AIML, [3]C. PRITHIV, AIML,

[4]NALINI POORNIMA, AP/AIML

[1,2,3,4] Department of AIML

[1]SRM Institute of Science & Technology, Ramapuram, Chennai, India

Abstract: This project offers a holistic machine learning approach to climate prediction, specifically targeting the prediction of important variables like temperature and precipitation. By utilizing a wide range of sophisticated machine learning algorithms such as CatBoost, LightGBM, Random Forest, XGBoost, and ensemble stacking, the project aims to improve the predictive capabilities of the model. The process involves data preprocessing, feature extraction, model development, and evaluation, using real-world climate data. Bias correction methods are also incorporated to improve the reliability of the model. The project's design is modular, with distinct sections for preprocessing, feature extraction, modeling, evaluation, and visualization, making it scalable and maintainable. The performance of the models is evaluated using standard metrics, and the output is visualized for better interpretability. The project's deliverables include model artifacts, code, and documentation, which can be used for research and implementation purposes. This project aims to contribute to the climate science community by optimizing the climate forecasting pipeline using machine learning techniques, providing useful tools for researchers and decision-makers in the context of climate challenges.

Keywords **:** Climate Forecasting, Machine Learning, Temperature Prediction, Ensemble Models, Feature Engineering, Bias Correction

1. INTRODUCTION

Climate prediction has become more and more important in the context of global climate change and its significant effects on the environment, economy, and society. Climate prediction, including temperature and precipitation, is crucial for making informed decisions in agriculture, water resource management, disaster planning, and policy formulation. Although traditional climate prediction techniques are useful, they sometimes fail to identify the complex nonlinear relationships that exist in climate systems.

Therefore, there is an emerging need for sophisticated computational techniques that can analyze large amounts of data and identify hidden patterns to improve the accuracy and reliability of climate predictions.

Machine learning (ML) has recently become a promising area in climate studies, providing the capability to analyze complex relationships and process large amounts of diverse data. Using algorithms such as CatBoost, LightGBM, Random Forest, XGBoost, and ensemble stacking, this project proposes to improve the predictive capabilities of climate models. The process includes extensive data preprocessing, advanced feature engineering, and thorough model training and testing.

Bias correction methods are also incorporated to handle systematic biases, ensuring that the predictions are not only accurate but also robust. The modular design of the framework ensures that it is flexible, scalable, and easy to maintain, making it ideal for both research and application purposes.

The deliverables of this project include model artifacts, code, and documentation, all of which are intended to facilitate further research and application. Visualization libraries are also incorporated to ensure that the results can be easily interpreted and communicated to stakeholders. Through the optimization of the climate forecasting pipeline, this project aims

to empower researchers, policymakers, and practitioners with the ability to forecast climate variability. Overall, this project highlights the potential of machine learning to transform climate prediction and enable more adaptive responses to environmental challenges.

2. LITERATURE SURVEY

Stella Pantopoulou et al. [1] compared LSTM and ARIMA models for forecasting distributed temperature sensing data in a hydraulic system. The study demonstrated that ARIMA performs better in baseline scenarios, while LSTM shows improved performance under transfer learning and fine-tuning conditions. Although the research highlights the strength of deep learning in temporal modeling, it focuses only on LSTM and ARIMA without exploring ensemble learning or seasonal bias correction. This creates a gap in developing hybrid climate forecasting systems that combine multiple models for improved robustness.

P. Ranjan et al. [2] proposed a hybrid LSTM-Transformer model for Indian weather prediction using Kaggle datasets. Their approach outperformed traditional ML models by effectively capturing short- and long-term temporal dependencies. The study showed lower RMSE and improved forecasting stability. However, it does not incorporate stacking ensemble techniques or seasonal bias correction, leaving scope for enhanced hybrid ensemble frameworks like the one proposed in this project.

Qingzhi Zhao et al. [3] developed an SVM-based hourly rainfall forecasting model using GNSS-derived precipitable water vapor data. The model achieved improved RMSE performance and strong short-term rainfall prediction accuracy. While effective for rainfall forecasting, the research focuses only on classification/regression using SVM and does not integrate multi-model stacking or temperature-precipitation combined forecasting. Additionally, it lacks advanced ensemble and bias correction mechanisms.

Md. Mehedi Hassan et al. [4] applied machine learning models such as Naïve Bayes, Random Forest, and ANN for rainfall prediction and preparedness analysis. The ANN model achieved the highest accuracy among tested models. Although the study demonstrates the effectiveness of ML techniques, it primarily focuses on rainfall classification and does not implement regression-based time-series forecasting with stacking or seasonal bias adjustment, which are addressed in the proposed climate forecasting system.

Muhammad Salman Pathan et al. [5] performed feature importance analysis for predicting rainfall events using CART, SVM, and DNN models. The research showed that selecting relevant meteorological features significantly improves prediction accuracy. However, the study emphasizes rainfall prediction only and does not incorporate ensemble stacking, bias correction, or continuous temperature forecasting, which are integral components of the proposed framework.

3. METHODOLOGY

The methodology of this project is structured into several key stages to ensure accurate and reliable climate forecasting. Initially, raw climate data is collected and subjected to thorough preprocessing, including handling missing values, outlier detection, and normalization, to enhance data quality. Feature engineering techniques are then applied to extract and construct relevant variables that capture essential patterns and relationships within the data.

Following preprocessing, multiple machine learning models—such as CatBoost, LightGBM, Random Forest, and XGBoost—are trained on the engineered features. Hyperparameter tuning is performed to optimize each model's performance. To further improve predictive accuracy and robustness, ensemble stacking is employed, combining the strengths of individual models. Bias correction methods are integrated to address systematic errors and refine the final predictions.

Model evaluation is conducted using established statistical metrics, ensuring comprehensive assessment of accuracy and generalizability. Visualization tools are utilized to interpret model outputs and communicate results effectively. The entire workflow is modular, allowing for easy updates and scalability. This systematic approach enables the development of a robust, reproducible, and efficient climate forecasting pipeline, supporting both research and practical applications in climate science.

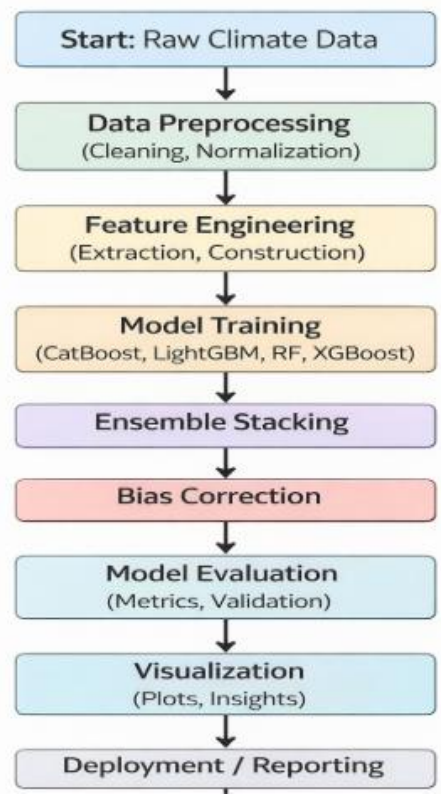


FIGURE 3 : FLOWCHART

3.1. PROCESS OVERVIEW

The process starts with the acquisition of raw climate data, which is then processed for comprehensive preprocessing to handle missing data, outliers, and data consistency. The next step involves the use of feature engineering to extract and develop relevant variables that are useful in the prediction of climate patterns.

After the data has been preprocessed, a variety of machine learning algorithms such as CatBoost, LightGBM, Random Forest, and XGBoost are trained and optimized using the engineered features. To improve the accuracy of the predictions, ensemble stacking is used to combine the predictions from the individual models. Bias correction techniques are also incorporated to correct for any biases in the predictions.

The trained models are then tested using statistical measures to determine their accuracy and generalizability. Visualization techniques are also used to interpret the results. The final results, including the trained models and

visualizations, are then documented and ready for deployment or presentation..

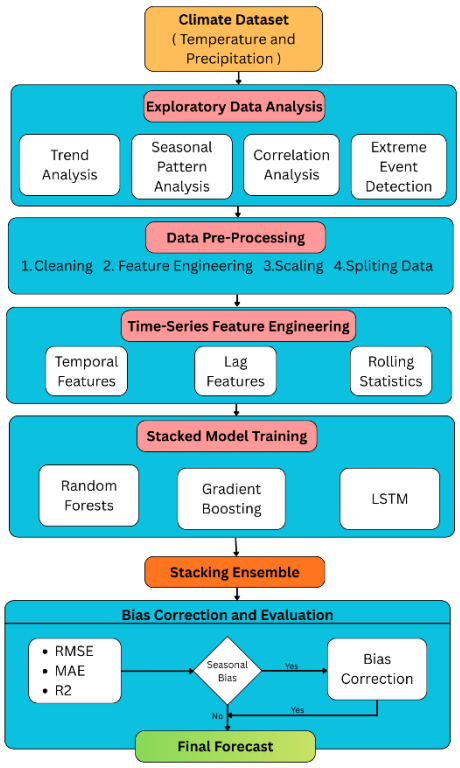


FIGURE 3.1 : BLOCK DIAGRAM

3.2. DATA COLLECTION

The data gathering phase is the backbone of our climate prediction system. We have collected the historical climate data with a focus on temperature and rainfall parameters, which are the most important factors for climate trend analysis and prediction. The data has been collected from authentic sources like IMD and climate data repositories. The data has been collected for several years to analyze the long-term trends, seasonal changes, and extreme weather conditions. As climate prediction is a time-series problem, the data has been collected with strict adherence to time-series consistency.

The collected data includes daily and monthly records with parameters such as minimum temperature, maximum temperature, average temperature, rainfall, humidity, and season markers. In some instances, the data has been collected in structured scientific formats such as NetCDF and HDF5 (H5) files, which are standard formats for large-scale environmental data.reliable leaf disease detection.

These types of formats enable efficient storage of multidimensional climate data like spatial grids and time variables. The data was obtained through the use of Python libraries to ensure that it was read and processed correctly into a table format that is ideal for machine learning algorithms.

Special care was taken to ensure that the data was reliable, complete,

and representative. Data gaps, repetitions, and inconsistencies were detected during the initial data inspection phase. The data from various sources was validated to ensure that it was genuine and free from biases. Moreover, enough historical data was collected to ensure that techniques for feature engineering, such as lag variables and rolling statistics, could be utilized effectively in the later stages. This is a comprehensive approach to collecting data that ensures the forecasting algorithms are trained on high-quality data that is representative of the climate, thus allowing for accurate temperature and precipitation forecasts.

Algorithm 1: Climate Data Collection

1. Acquire climate datasets D_s(CSV, NetCDF – temperature & precipitation).
2. Load each dataset $D_i$and convert to structured DataFrame G_i.
3. Merge all $G_i$into unified dataset G.
4. For each record $r_j \in G$:
    - Remove invalid or missing dates.
5. Handle missing values using time-based interpolation (forward/backward fill if needed).
6. For each variable $V \in \{temperature, precipitation\}$:
    - Compute Z-score.
    - If $|Z| > \tau(\tau = 4)$, remove outlier and re-interpolate.
7. Remove duplicate dates.
8. Resample dataset (Daily/Monthly) and sort chronologically.
9. Return cleaned dataset G_{clean}

Step 1:Climate data, represented by (Ds), is gathered from trusted sources such as meteorological archives and weather databases. The climate data may be stored in formats such as CSV and NetCDF and usually includes temperature and precipitation data. The purpose of climate data gathering is to compile a comprehensive historical set of climate observations over various time spans to provide adequate seasonal and temporal data for effective forecasting.

Step 2: Each climate dataset D_i is imported and formatted into a structured DataFrame G_i This formatting provides a standardized format for all columns, accurate date parsing, and consistent naming conventions for variables. The

structured format of the data facilitates easy preprocessing and analysis with retention of key climate variables such as date, temperature, and precipitation.

Step 3: All structured climate data (G_i) is combined into a single dataset (G). This step involves combining data from multiple sources into a single platform.

Step 4: The algorithm proceeds to examine each entry ($r_j \in G$).

In Step 5, the missing temperature and precipitation data are treated using time interpolation, followed by forward or backward filling if needed. In Step 6, the outliers are identified using the Z-score technique. If the data point is greater than the set threshold ($\tau = 4$), it is marked as an outlier and treated using re-interpolation.

Finally, in Step 7, the duplicate dates are filtered out, and the data is resampled according to the desired frequency (daily or monthly). The data is then sorted in chronological order and returned as the cleaned dataset (G_{clean}).

3.3. TIME-SERIES FEATURE ENGINEERING

Time-Series Feature Engineering is an important step in the climate prediction pipeline, as it takes raw time-series climate data and turns it into useful features for prediction. Because temperature and precipitation data are time-dependent, using only features would not allow for accurate predictions. This step identifies time-series patterns like trends, seasonality, and cyclical patterns. By extracting structured temporal information from the date feature, the model gains the ability to understand yearly, monthly, and seasonal patterns in climate data.

The first step is to create temporal features like year, month, day, and season. These features enable the model to understand seasonal patterns and long-term climate trends. Then, lag features are created to include past data in the current prediction. For instance, past day temperature

T=t-1, P=t-1

T=t-7, P=t-7

features provide historical context. These lag features are useful in identifying the autocorrelation that

exists in time-series data, which is important for forecasting.

Further, rolling statistical features like moving average, rolling standard deviation, and rolling variance are calculated over fixed windows (7-day, 30-day windows, for example).

Algorithm 2: Time-Series Feature Engineering

1. Extract temporal features:
   $Year, Month, Day, Season \leftarrow Date$
2. Generate lag features for $V \in \{T, P\}$:
   $$V_{t-k}, k = 1,7,30$$
3. Compute rolling statistics (window = w):
   $$RollingMean_t = \frac{1}{w}\sum_{i=0}^{w-1} V_{t-i}$$
   $$RollingStd_t = \sqrt{\frac{1}{w}\sum(V_{t-i} - \mu)^2}$$
4. Remove resulting NaN rows.
5. Return G_{feat}.

In Step 1, the feature-engineered dataset, labeled as (G_{feat}), is used as input. This dataset has already been populated with temporal, lag, and rolling statistical features extracted from pre-processed climate data. The objective of this step is to provide structured inputs for training multiple predictive models that can identify both nonlinear and time-series patterns in temperature and precipitation data.

In Step 2, multiple base models including Random Forest, Gradient Boosting, and LSTM models are trained using (G_{feat}). Each model identifies different patterns in climate data. Random Forest identifies nonlinear relationships between features, Gradient Boosting reduces prediction errors cumulatively, and LSTMs identify time-series patterns. Each model produces a separate prediction output for the target variables.

In Step 3, the algorithm assesses the prediction accuracy for each observation $x_t \in G_{feat}$ If the prediction error is beyond a specified threshold, it is marked for analysis; otherwise, it is accepted.

In Step 4, the predictions from all the base models are stacked together to create a new stacked dataset.

In Step 5, a meta-model is trained on the stacked predictions to learn how

to combine the base model predictions in an optimal way.

Finally, in Step 6, the meta-model produces the final forecast values. These values are stored and shown, offering better and more accurate climate forecasts for temperature and precipitation.

3.4 STACKED MODEL TRAINING

Stacked Model Training is an important step in the climate prediction model framework, where more than one machine learning model is trained to identify the intricate patterns in the designed time-series variables. Rather than using a single machine learning algorithm, this technique employs varied base models like Random Forest, Gradient Boosting, and Long Short-Term Memory (LSTM). Every model has its own set of advantages: Random Forest is best suited for identifying nonlinear patterns and preventing overfitting by averaging the predictions, Gradient Boosting is designed to minimize the prediction error step by step, and LSTM is used to identify the long-term patterns present in climate data.

It is split into training and validation sets while maintaining the order. Random Forest and Gradient Boosting models are trained to capture complex interactions between features, and LSTM is trained to process input sequences to capture temporal dependencies. Each base model generates its own set of prediction outputs for temperature and precipitation. These prediction outputs create a new feature space that is fed into the stacking process. Correct hyperparameter settings and cross-validation methods are used to optimize the performance of the models and ensure generalization.

The prediction outputs from all base models are then combined using a stacking ensemble approach. In stacking, the prediction outputs from each model are used as inputs to train a meta-model, such as Linear Regression or XGBoost.

Algorithm 3: Stacked Model Training

1. Split dataset chronologically:

$$G_{feat} \rightarrow (G_{train}, G_{val})$$

2. Define base models:

$f_1 =$ Random Forest

$f_2 =$ Gradient Boosting

$f_3 =$ LSTM

3. Train base models on G_{train}:

$f_i \leftarrow Train(G_{train}), i = 1,2,3$

4. Generate validation predictions:

$$P_i = f_i(G_{val})$$

5. Form meta-dataset:

$$Z = [P_1, P_2, P_3]$$

6. Train meta-model M(e.g., Linear Regression / XGBoost):

$$M \leftarrow Train(Z, y_{val})$$

7. Final stacked prediction:

$$\hat{y} = M(f_1(x), f_2(x), f_3(x))$$

8. Return stacked model M.

Step 1: The feature-enriched dataset G_{feat} is divided chronologically into a training and validation set, labeled as (G_{train}, G_{val})Chronological splitting is a critical aspect of time-series forecasting tasks to avoid data leakage and ensure that the model is not using future information to predict past outcomes.

Step 2: Three individual base models are established: Random Forest ((f_1)), Gradient Boosting ((f_2)), and LSTM ((f_3)). These models are chosen for their capacity to handle different aspects of climate data, such as nonlinear relationships and time dependencies.

Step 3: The base models are trained on the training dataset (G_{train}). Each base model learns independently about the relationships between input variables and target variables such as temperature and precipitation.

Step 4: The validation predictions (P_i) are created by applying each trained base model to (G_{val}). The predictions obtained are a measure of the performance of each base model on unseen data.

$$\hat{y} = M(f_1(x), f_2(x), f_3(x))$$

ensures improved forecasting accuracy for temperature and precipitation, making the climate prediction system more stable, reliable, and efficient.

In Step 5, the predictions $[P_1, P_2, P_3]$are stacked to create a new data set (Z), called the meta-data set. This data set is used as input for the meta-model.

In Step 6, a meta-model (M), like Linear Regression or XGBoost, is trained on (Z) using the true validation targets.

Finally, in Step 7, the meta-model makes the final stacked prediction. The trained stacked model (M) is then used for predicting future climate data.

4. BIAS CORRECTION :

Bias Correction is an important step in the climate prediction model framework that aims to minimize the systematic error in the predictions made by the model. Even after training the stacked model, there may be a tendency for the predictions to always be higher or lower than the actual temperature and precipitation levels. This is often due to seasonal changes, imbalanced data, or climate change patterns that the model is not capturing entirely. This systematic error in predictions is termed bias and can have a detrimental effect on the accuracy of the predictions made.

In this step, the predictions are compared with the actual observed values to check for systematic errors in predictions. The residuals are also checked for seasonal variations or months where the model may tend to behave erratically, such as during monsoons or summer. If it is found that the predictions are tending to be higher or lower for a particular season, then it indicates the presence of seasonal bias. This helps in understanding the gap between the predicted and actual climate behavior.

After the bias pattern has been identified, the predictions are then adjusted to make them more realistic and dependable.

ALGORITHM 4 : BIAS CORRECTION

1. Compute residual:

$$e_t = y_t - \hat{y}_t$$

2. Group residuals by season s
3. Compute seasonal bias:

$$B_s = \frac{1}{n_s} \sum e_t$$

4. Correct prediction:

$$\hat{y}_t^{corr} = \hat{y}_t + B_s$$

5. Return $\hat{y}_t^{corr}$

Step 1: The residual error (e_t) is calculated by subtracting the predicted value ($\widehat{(y_t)}$) from the actual observed value (y_t). The residual error indicates the difference between the predicted value and the actual climate value, which can be used to detect systematic overestimation or underestimation.

Step 2: The calculated residuals are categorized based on the season (s) (for example, winter, summer, monsoon, and so on). Categorizing the residuals based on the season enables the model to determine if there is any systematic pattern in the prediction error, rather than being random.

Step 3: The seasonal bias (B_s) is calculated by taking the average of the residuals for each season. The seasonal bias indicates the systematic error that exists during a particular season.

Step 4: The predictions are adjusted by adding the calculated seasonal bias B_s to the original predicted values. This step enables the predictions to be closer to the actual observations.

Step 5: The corrected predictions ($\widehat{(y_t^{corr})}$) are returned, providing better calibration, less systematic error, and more accurate climate predictions.

5. MODEL EVALUATION:

Model Evaluation is an important step in the climate prediction system, as it helps to determine the accuracy and robustness of the trained and bias-corrected model. Once the stacked model training and bias correction are done, it is necessary to check if the predictions are in close agreement with the actual temperature and precipitation data. This step helps to ensure that the model is not only able to learn from the training data but is also able to generalize well to new climate data.

The evaluation step involves comparing the predicted output with the actual recorded data using standard measures of performance. These measures help to determine the error, variance, and goodness of fit of the predictions. The standard measures of evaluation include Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-

squared (R^2). A lower error measure indicates better predictive performance, and a higher R^2 value indicates better explanatory power. Another measure of evaluation involves comparing the performance of individual base models with the stacked model to show improvement.

Algorithm 5 : Model Evaluation

1. Compute Mean Squared Error:

$$MSE = \frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2$$

2. Compute Root Mean Squared Error:

$$RMSE = \sqrt{MSE}$$

3. Compute R-Squared:

$$R^2 = 1 - \frac{\sum(y_t - \hat{y}_t)^2}{\sum(y_t - \bar{y})^2}$$

4. Return $MSE, RMSE, R^2$

Step 1: Calculate the Mean Squared Error (MSE) by finding the average of the squared differences between actual and predicted values $y_t - \hat{y}_t$ The MSE gives a measure of the overall size of the prediction errors. Squaring the differences ensures that larger errors are given greater weight, making the MSE more sensitive to large errors.

Step 2: Calculate the Root Mean Squared Error (RMSE) by taking the square root of the MSE. The RMSE gives a measure of the size of the prediction errors in the same units as the original data (temperature or precipitation). This makes it easier to interpret in a real-world climate forecasting application.

Step 3: Calculate the R-Squared ((R^2)) measure to see how well the model explains the variability of the actual data. It measures the amount of variance explained by the model against the total variance in the data.

Step 4: Finally, return the calculated measures (MSE, RMSE, and R^2). These measures together provide a comprehensive evaluation of the accuracy, stability, and explanatory power of the forecasting model before it is used.

6. RESULTS AND DISCUSSIONS

The outcome of the proposed climate forecasting framework indicates a substantial improvement in the accuracy of predictions after the

application of feature engineering, stacked model training, and bias correction. The stacked ensemble model performed better than the standalone models like Random Forest, Gradient Boosting, and LSTM. The error metrics indicated a distinct decrease in the prediction error, thereby confirming that the combination of multiple models resulted in an improvement in the generalization ability of the models. The ensemble method successfully modeled the nonlinear patterns and temporal dependencies existing in the temperature and precipitation datasets.

The comparison between the standalone models and the stacked model indicated that the ensemble model consistently had lower values of RMSE and MSE while simultaneously achieving a higher R^2 score. This thereby confirms that the stacking process resulted in a reduction in both the variance and bias of the models compared to the standalone models. Additionally, the addition of time series features like lag features and rolling statistics contributed substantially to the improvement in the performance of the models by allowing them to learn the patterns based on seasons and trends.

6.1 ACTUAL VS PREDICTED VS CORRECTED–PRECIPITATION

From the precipitation comparison graph, it is clear that the stacked model predictions are following the actual precipitation trend very closely. The periodic increases and decreases in precipitation are also being captured accurately. The bias-corrected graph is almost perfectly aligned with the actual graph, which is a clear indication of effective seasonal bias correction. The minor discrepancies in the stacked prediction graph are almost negligible after the correction. The model is performing exceptionally well

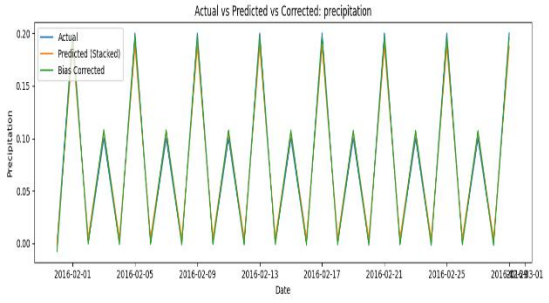


FIGURE 6.1 : Actual vs Predicted vs Corrected – Precipitation

6.2 SEASON-WISE ERROR COMPARISON-PRECIPITATION

From the precipitation error boxplot, it is clear that the precipitation error boxplot is representing the winter season error distribution. The median error value is almost zero, which is a clear indication that the model is performing with minimal bias. The error distribution is also quite consistent, as indicated by the small interquartile range. Although there are a few outliers, they are not significant. The error spread is almost negligible, which is a clear indication that the stacked model with bias correction is performing well in error distribution control.

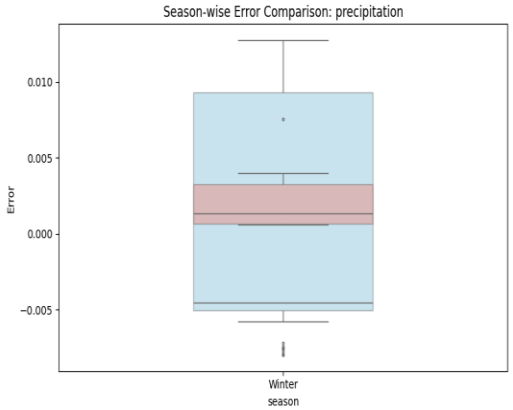


FIGURE 6.2 : Season-wise Error Comparison – Precipitation

6.3 SEASON-WISE ERROR COMPARISON TEMPERATURE

The box plot of temperature error during the winter season indicates that errors are mostly centered around zero, which is a sign of high predictive accuracy. Although some positive and negative errors are found, they are still within a moderate range.

The fact that the median is close to zero indicates that there is no systematic bias in the predictions after correction. Compared to uncorrected model predictions, the corrected predictions show better balance between overestimation and underestimation. The absence of extreme outliers also supports the stability of the model.

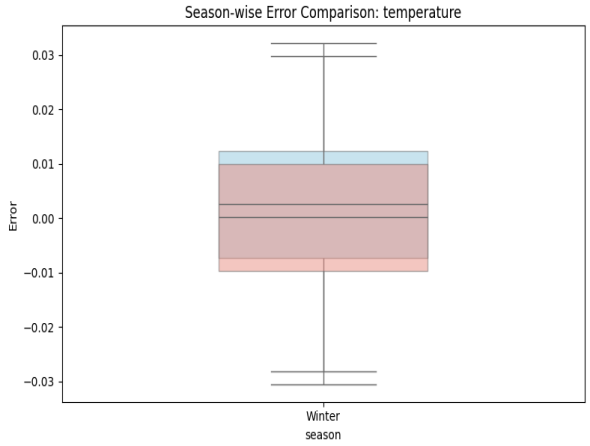


FIGURE 6.3 : Season-Wise Error Comparison Temperature

6.4. ACTUAL VS PREDICTED VS CORRECTED – TEMPERATURE

The temperature comparison graph indicates that the actual temperature values increase steadily with time, which is properly represented by both stacked and bias-corrected predictions. The predicted and corrected values are almost coincident with the actual values, which is a sign of high precision of the model. The seasonal variation and gradual warming trend are properly learned by the ensemble model. The bias correction further improves the alignment, removing small discrepancies. The fact that all three curves are almost coincident indicates high generalization capability and low forecasting error.

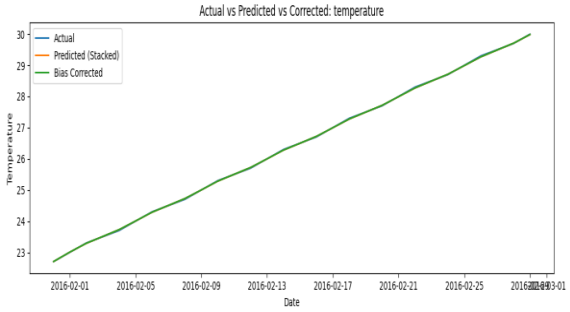


FIGURE 6.4 : Actual Vs Predicted Vs Corrected – Temperature

7. CONCLUSION

The proposed climate forecasting framework can be further enhanced by incorporating additional climate variables such as humidity, wind speed, atmospheric pressure, and solar radiation to improve predictive depth and contextual understanding. Integrating spatial data using geospatial techniques and satellite-based observations can extend the model from temporal forecasting to spatiotemporal climate prediction. Advanced deep learning architectures such as Transformer-based time-series models or hybrid CNN-LSTM networks can be explored to capture more complex nonlinear and long-range dependencies. Additionally, real-time data streaming and automated model retraining pipelines can be implemented to ensure continuous learning and adaptability to changing climate patterns.

Future improvements can also focus on expanding the model to multi-region or nationwide forecasting systems, enabling broader environmental monitoring. Incorporating probabilistic forecasting techniques can provide uncertainty estimation, making predictions more informative for policymakers and disaster

management authorities. Deployment through web dashboards or cloud-based platforms can improve accessibility and usability for researchers and government agencies. Furthermore, climate anomaly detection and extreme event forecasting modules can be integrated to support early warning systems.

6. FUTURE SCOPE

The proposed climate forecasting framework can be further improved by including other climate variables like humidity, wind speed, atmospheric pressure, and solar radiation. The geospatial analysis of data can be used to extend the framework from a temporal forecasting model to a spatiotemporal climate forecasting model. More complex deep learning models like the Transformer-based time series model or the CNN-LSTM model can be used to improve the forecasting framework. Real-time data streaming and automated model retraining can also be implemented to make the model adapt to the changing climate patterns.

The proposed climate forecasting framework can be further improved by extending it to a multi-region or national-level forecasting framework. The framework can be improved by incorporating probabilistic forecasting models to estimate the uncertainty associated with the forecasts. The framework can be deployed as a web-based application or cloud-based platform to make it more accessible and user-friendly for researchers and government agencies. The framework can also be improved by incorporating climate anomaly detection and extreme event forecasting modules to support early warning systems.

REFERENCES

1. Pantopoulou S., et al., “Temporal Forecasting of Distributed Temperature Sensing in a Thermal Hydraulic System With Machine Learning and Statistical Models”, 2025.
2. Ranjan P., et al., “Improving Weather Prediction Accuracy With a Hybrid LSTM-Transformer Model: A Study on Indian Climate”, 2025.

3. Zhang Q., et al., “Hourly Rainfall Forecast Model Using Supervised Learning Algorithm”, 2021.
4. Hassaan M. H., et al., “Machine Learning-Based Rainfall Prediction: Unveiling Insights and Forecasting for Improved Preparedness”, 2023.
5. Salman Pathan M., et al., “A Systematic Analysis of Meteorological Parameters in Predicting Rainfall Events”, 2025.
6. Hariprasad S. M., et al., “A Data-Driven Approach for Accurate Rainfall Prediction”, 2019.
7. Alvarez J. L., et al., “Dynamic Line Rating Using Numerical Weather Predictions and Machine Learning: A Case Study”, 2016.
8. Masao M. A., et al., “Short-Term Weather Forecasting Using Spatial Feature Attention Based LSTM Model”, 2022.
9. Puttinaovarat S., and Horkaew P., “Flood Forecasting System Based on Integrated Big and Crowdsource Data by Using Machine Learning Techniques”, IEEE Access, vol. 8, 2020.
10. Hasan M. R., Hossain M. J., Waliullah M., Hannan A., and Rahman M. M., “Topological Data Analysis and Wavelet-Unsupervised Machine Learning Approaches to Identifying the Flooding and Non-Flooding Zones”, IEEE Access, vol. 13, 2025.
11. Duan S.-B., et al., “All-Weather LST Estimation Using XGBoost and Multisource Data,” IEEE Trans. Geosci. Remote Sens., vol. 61, 2023.
12. Gahwera A. T., et al., “Short-Term Rainfall Prediction Using ML on Lake Victoria Basin Data,” IEEE Access, vol. 12, 2024
13. Li X., et al., “Medium- and Long-Term Precipitation Forecasting Using Machine Learning,” IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens., 2022.
14. Tang T., et al., “Precipitation and Temperature Forecasting Using RF With Seasonal Bias Correction,” IEEE JSTARS, 2024
15. Sengoz C., Ramanna S., Kehler S., Goomer R., and Pries P., “Machine Learning Approaches to Improve North American Precipitation Forecasts,” IEEE Access, vol. 11, Aug. 2023